Gridworld Agent Report

Group 8:
Ryan Middleton, Owen Sawler, Lawrence Onyiuke, Norah Jurdjevic

Queen's University: CISC474

Introduction and Problem Formulation

In this problem we are tasked with navigating a 10x10 grid world which includes an agent, walls and guards which can see 4 tiles in each direction, rotating counter-clockwise up, left, down and right. The agent's goal is coverage, to navigate to each tile without being detected by the guards or running out of moves. The game ends when the agent covers all tiles, reaches the maximum of 500 moves or is caught by the guards. An example map is shown in figure 1 below, where white and pink tiles have already been covered, red and pink tiles are observed by a guard, green cells are the guards themselves, brown cells are walls and black cells are uncovered.

We approach this problem using Reinforcement Learning methods. In order to solve this problem we must formally define the state space, action space and reward scheme. This project emphasizes the importance of careful reward shaping and observation design, as small changes in these components can significantly impact agent behavior and overall performance.

Project Github: https://github.com/Owenn04/CISC-474-Final-Project

State and Observation Space

This problem has a discrete state space, with 100 tiles which have 6 possible values and an agent.This allows us to calculate the state space, taking 100 tiles with 6 possible values, and the possibility of one agent in each of the 100 tiles yields a total state space of 6^{100} x 100

Tile	Colour	Navigable
Wall	Brown	No
Guard	Green	No
Uncovered, Viewed by Guard	Red	Yes
Covered Tile, Viewed by Guard	Pink	Yes
Uncovered Tile	Black	Yes
Covered Tile	White	Yes

We devised multiple observation spaces throughout the development of this program:

- *FULL_GRID_OBSERVATION*:
    - Provides a flattened vector of the grid, this is the initial observation space
- *COMPACT_OBSERVATION*:
    - Smaller observation to reduce dimensionality. Includes agent row, agent column, percentage of cells covered, percentage of cells uncovered, percentage of steps remaining, percentage of cells which are enemies, and a value indicating if the game is over
- *HYBRID_OBSERVATION*

    - Includes all components of *FULL_GRID_OBSERVATION* and *COMPACT_OBSERVATION* and adds local action features
    - Local action features includes move validity, novelty and risk in each direction

After this we changed our approach, initially we attempted to find a complete observation and reward scheme which could complete complex maps such as *maze* and *chokepoint* but repeatedly fell short. These models could produce somewhat decent results, but would fail in completing basic maps. In an effort to build an agent which could achieve a successful mission, we majorly simplified our observation and reward scheme, then attempted basic levels like *just_go* and *maze*. This approach allowed us to iterate functional models.

- *BASELINE_OBS_V1*
    - Includes agent row, agent column, percentage of cells covered, percentage of steps remaining and move validity and novelty for each direction.
    - This observation space allowed us to complete *just_go*
- *BASELINE_OBS_V2*
    - Includes all features from v1, plus the percentage of uncovered cells in each direction from the agent
    - This observation space allowed us to complete *safe*
- *BASELINE_OBS_V3*
    - This includes all features from v2, plus an indicator if the adjacent tiles are viewed by the guards
    - This observation space wasn't sufficient to consistently complete *maze*, but did make improvements
- *BASELINE_OBS_V4*
    - This includes all features from v3, plus an value which forecasts guard field of view (FOV) cells, understanding the guards orientation
    - This observation space allowed us to complete *maze* and *chokepoint*
    - This is the observation space used in our best submitted model.

In order to implement many of these advanced observation features, we had to augment the provided environment at runtime. The env.py file which cannot be modified, provides a limited set of basic facts in the *info* dictionary. We take this information and parse it to gain access to a new understanding and set of features, and store a history of particular attributes such as agent position, and frontier distance history.

Action Space

The action space consists of 5 actions: [Left, Down, Right, Up, Stay]
Each has a corresponding numeric action ID: [0, 1, 2, 3, 4].

Reward Scheme

In a similar timeline to the observation spaces, we started with an initial batch of reward schemes which we used to conduct some initial testing, then simplified our approach and iterated to achieve our best results.

	SPARCE_REWARD	*COVERAGE_REWARD*	*SAFETY_REWARD*
Mission Success	+1.0	+3.0	+4.0
Game Over	-1.0	-2.0	-3.0
Step Penalty (Base)	0	-0.1	-0.02
New Cell Covered	0	+1.0	+0.75
Stayed Still	0	-0.05	-0.05
Move Blocked	0	-0.1	-0.1
In Enemy FOV	0	0	-0.25
All Cells Cleared	0	0	+0.5

BASELINE_REWARD…	*_V1*	*_V2*	*_V3*	*_V4*
Step Penalty (Base)	-0.01	-0.01	-0.03	-0.03
New Cell Covered	+1.0	+1.0	+1.0	+1.0
Move Blocked	-0.05	-0.05	-0.05	-0.05
Stayed Still	-0.05	-0.05	-0.05	-0.05
Mission Success	+50.0	+50.0	+50.0	+50.0
Two Step Oscillation	-0.2	-0.2	-0.2	-0.2
In Enemy FOV	0	-0.1	-0.1	-0.1
Game Over	0	-4.0	-4.0	-4.0
If Cells Remaining <= 10: New Cell Covered	0	0	+0.2	+0.2
If Cells Remaining <= 10: Else	0	0	-0.1	-0.08

If steps remaining <= 0: Game Over	0	0	-2.0 x Cells Remaining	-2.0 x Cells Remaining
Frontier Distance System	Not Included	Not Included	Not Included	Included

The frontier distance system was made to help the agent navigate towards uncovered cells and prevent stalling which was a recurring problem during training. A frontier cell is any uncovered tile. This system works by first identifying how many steps away the closest uncovered tile is using a simple breadth-first search from the agent's current position. When this distance decreases, the agent gets a small reward, when the distance gets larger, the agent gets a small penalty and when the agent's distance doesn't change it gets a smaller penalty. We modify these rewards to be more dramatic when the agent has <= 10 cells remaining as this is when we see the most stalling.

	Cells Remaining > 10	Cells Remaining <= 10
Towards Frontier	+0.02	+0.12
Away from Frontier	-0.02	-0.12
Same Distance from Frontier	0	-0.04

Methodology and Algorithm Description

We explored two algorithms for training our models, Proximal Policy Optimization (PPO) and Deep Q-Networks (DQN). After a brief trial period we found PPO trained smoothly over time and was sufficient for our purposes. PPO builds from the Policy Gradient Theorem and Trust Region Policy Optimization (TRPO). TRPO is very expensive and constrained, PPO simplifies the calculation of KL. Policy gradient methods are often easier to implement, though this wasn't a problem for us as we used stable baselines. Instead of learning Q-values like DQN, PPO learns a policy directly, a probability distribution over actions. PPO is an actor-critic method which uses an actor policy to pick actions and a critic policy to value, estimate and train the actor. It clips updates so the policy can't change too drastically in one step.

We used the stable-baselines3 implementation of PPO with mostly default hyperparameters. This includes a learning rate of $3e^{-4}$, a discount factor of 0.99, GAE lambda of 0.95, clip range of 0.2, and rollout length of 2048 steps. However, we made a few modifications to the defaults. First, we set the entropy coefficient to 0.02 (default 0.0) to encourage the agent to maintain exploration throughout training rather than collapsing to a deterministic policy too early. This was important because the final tournament evaluates on unseen maps, so the agent needs to generalize behavior rather than overfit on memorized paths. Second, we increased the MLP network architecture from two hidden layers of 64 neurons to two layers of 256, giving the model more capacity to learn the relationship between observation features such as enemy FOV forecasts and safe movement decisions.

Training duration was also another important consideration. We experimented with 500,000, 1,000,000, 2,000,000, and 5,000,000 timestep training runs. At 500k steps the agent learned basic coverage behavior but struggled with endgame cleanup. This was where the agent would cover 95% of the grid efficiently but then oscillate between expired cells, unable to locate the remaining uncovered cells. At 2M steps, the agent showed markedly improved performance on difficult maps, both in reducing this oscillation behavior and in navigating enemy-guarded passages that required precise timing to cross safely.

Results

We evaluated nine model configurations spanning four observation spaces (obs_v1 through obs_v4) and four reward functions (reward_v1 through reward_v4), trained on maps of increasing difficulty. Each model was trained using our PPO algorithm. Models were evaluated over 5 deterministic episodes on their respective training maps.

Configuration	Map	Steps	Success	Coverage	Survival
obs_v1 / reward_v1	just_go	200k	100%	100%	100%
obs_v2 / reward_v1	safe	300k	100%	100%	100%
obs_v2 / reward_v2	safe	300k	0%	61%	100%
obs_v3 / reward_v2	maze/choke	300-650k	30%	56%	30%
obs_v3 / reward_v3	maze	650k	80%	92%	80%
obs_v4 / reward_v2	chokepoint	700k	40%	97%	100%
obs_v4 / reward_v3	chokepoint	400k	20%	97%	100%
obs_v4 / reward_v4	chokepoint	700k	60%	98%	100%

Table 1: Evaluation results for each observation/reward configuration.

Model Comparison

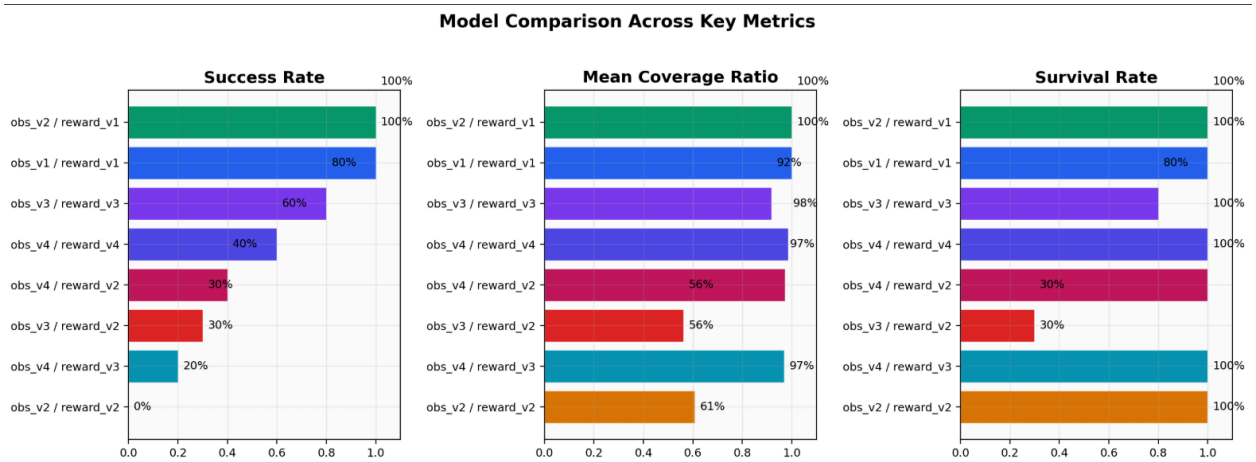

Figure 1: Side-by-side comparison of success rate, coverage ratio, and survival rate across all configurations. obs_v2/reward_v1 achieves the best overall performance (100% across all metrics) on the safe map, while obs_v4/reward_v4 achieves the best results on the hardest map (chokepoint) with 60%

success rate, 97% coverage, and 100% survival. The gap between obs_v3 and obs_v4 with the same reward_v2 highlights the impact of FOV forecasting due to obs_v4 97% coverage vs ovs_v3 56%.

Observation and Reward Cross-Comparison

The following heatmaps compare each observation/reward combination across key metrics. Each cell represents a unique configuration, with darker colours indicating better performance. Empty cells indicate combinations that were not tested.

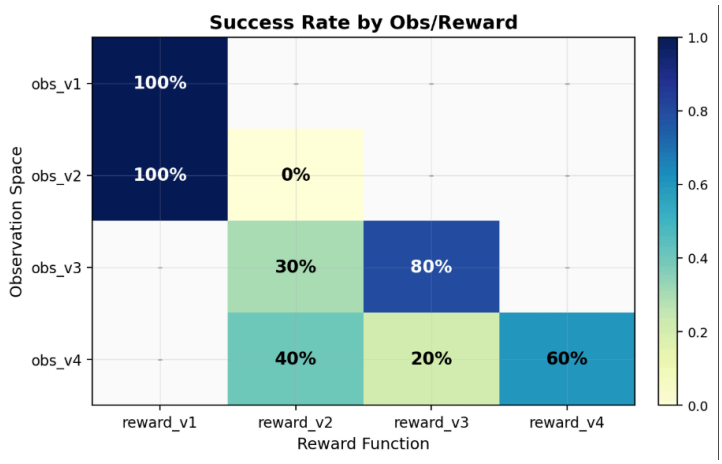

Figure 2: Success rate by observation and reward. The top-left corner (simple obs, simple reward, simple maps) achieves 100% success. Moving down and right (harder obs, harder reward, harder maps) indicate that the success rate decreases. obs_v4/reward_v4 achieves the highest success rate among the chokepoint configurations, demonstrating that frontier shaping in reward_v4 converts near-complete coverage into mission completion.

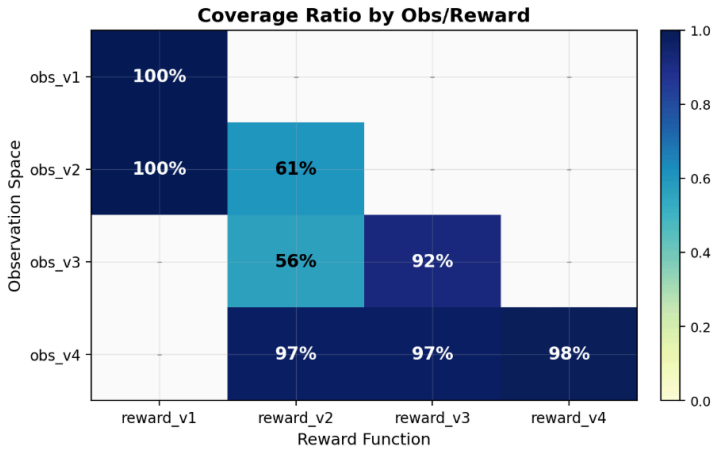

Figure 3: Average coverage_ratio. Obs_v4 consistently achieves 97-98% coverage regardless of reward mode, while obs_v3 shows much more sensitivity to reward choice (56% with reward_v2 vs 92% with

reward_v3). This demonstrates that the FOV forecasting features in obs_v4 are the primary driver of high coverage on enemy-heavy maps, with reward shaping playing a secondary role.

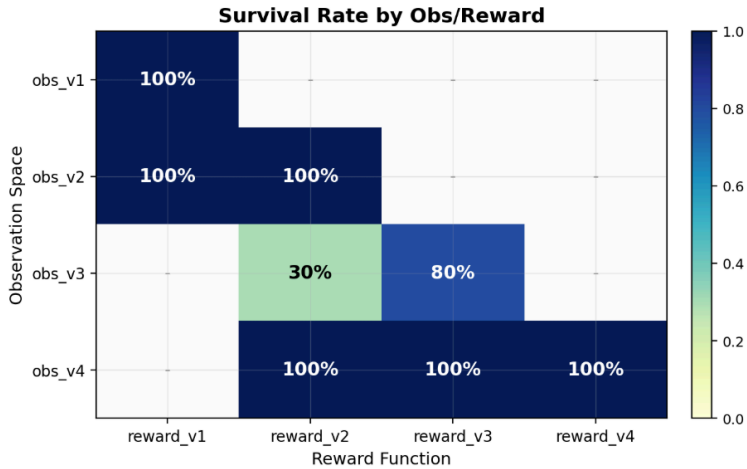

Figure 4: Survival rate (percentage of episodes where the agent was not detected). All obs_v4 configurations achieved 100% survival on chokepoint, meaning the agent perfectly avoids enemies but sometimes fails to complete full coverage within the step limit due to getting stuck. Obs_v3 has only a 30% survival rate with reward_V2, indicating without FOV forecasting, the enemy walks into enemy fields of view.

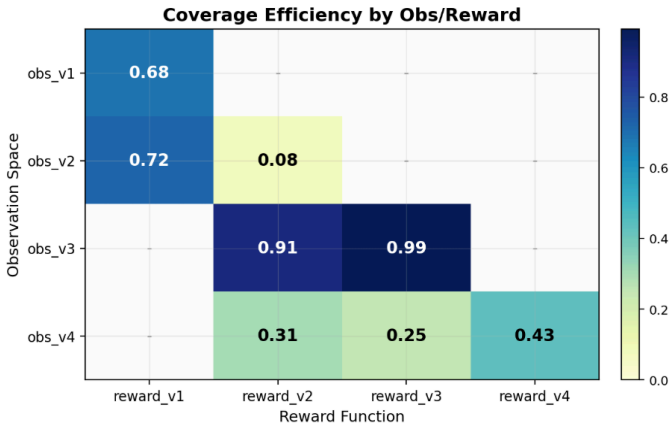

Figure 5: Coverage efficiency (cells covered per step). obs_v3/reward_v3 achieves the highest efficiency (0.99), meaning almost every step covers a new cell on the maze map. Obs_v4 configurations show lower efficiency (0.25-0.43) because the chokepoint map requires traversing already-explored territory to reach isolated sections. obs_v2/reward_v2 has the lowest efficiency (0.08) because the agent uses all 500 steps without completing the map.

Coverage vs Success Tradeoff

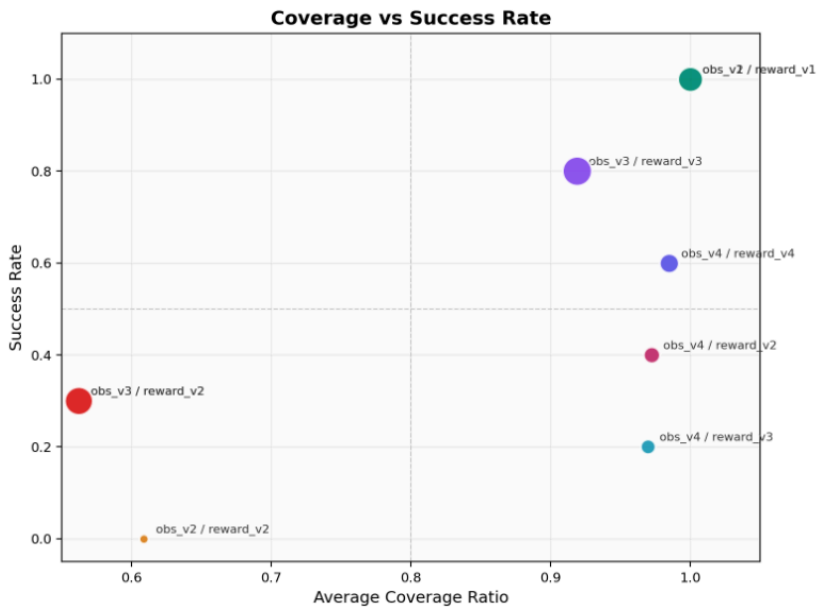

Figure 6: Coverage ratio vs success rate. Dot size indicates coverage efficiency. The ideal position is the top-right corner (high coverage, high success). obs_v2/reward_v1 sits at the top right on the simplest map. On harder maps, obs_v4/reward_v4 offers the best balance of high coverage (98%) and reasonable success (60%). obs_v3/reward_v2 (red, bottom-left) shows the failure of insufficient observation features on enemy maps. The large dot indicates high efficiency when alive but with low success and coverage due to frequent deaths.

Key Findings:

1. **Observation space improvements had a larger impact than reward shaping.** Holding reward_v2 constant, obs_v4 achieved 97% coverage on chokepoint while obs_v3 achieved only 56%. The FOV forecasting features in obs_v4 were the single most impactful addition, enabling the agent to time movement through enemy-guarded passages without being detected.
2. **Reward shaping primarily affects endgame completion, not coverage.** Across the obs_v4 row, all three reward modes achieved 97-98% coverage and 100% survival, but success rate varied from 20% to 60% across different reward schemes. The frontier distance shaping in reward_v4 guides the agent toward remaining uncovered cells during the endgame, converting near-complete coverage into actual success.
3. **Simple observations work well for simple maps.** Obs_v1 with just 12 features achieved 100% success rate on just_go. Obs_v2 with 16 features achieved 100% success on safe with reward_v1. Adding complexity to the observation only helps when the map demands it.
4. **Survival does not guarantee success.** All obs_v4 models achieved 100% survival on chokepoint but only a 20-60% success rate. The agent learned to avoid enemies perfectly but could not complete full coverage within the 500 step limit. This highlights endgame navigation as the primary remaining challenge.
5. **Reward function mismatch degrades performance.** Obs_v2 achieved 100% success with reward_v1 on safe but 0% success with reward_v2 on the same map. Reward_v2 includes enemy

FOV penalties and a death penalty which is irrelevant on maps with no enemies, producing a confusing training signal, preventing the agent from learning efficient coverage. The key take away is that to build the most robust model, a general map-set needs to be taken into account.

6. **Higher observation dimensionality trades efficiency for robustness.** Obs_v3 achieves 99% coverage per step on maze (highly efficient) but only 56% coverage on chokepoint. Obs_v4 achieves lower efficiency at 25-43% but higher and more consistent coverage at 97-98% across different maps and reward functions. The additional observation features make the agent more cautious, but instead more reliable.

Best Performing Model

Our best performing model is:
ppo_hard_generalization_mix_baseline_obs_v4_baseline_reward_v4_7500000.zip
It beats all maps except for "custom_challenge", "double_maze", "triple_patrol" and "sneaky_enemies" where it achieves great coverage then stalls.

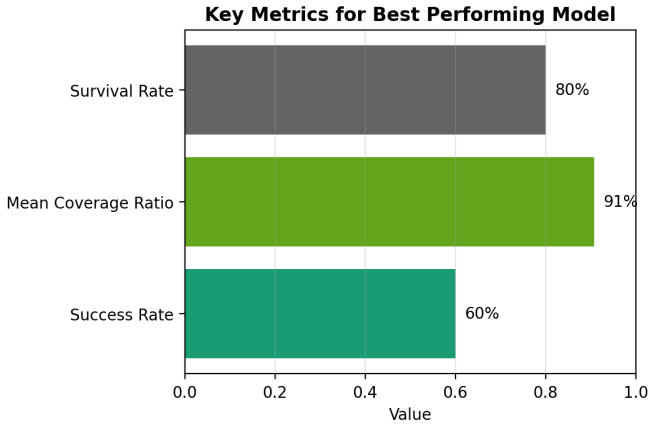

Figure 7: The survival rate, mean coverage ratio and success rate of the best performing model. The high survival rate and coverage ratio indicate that the model explores most of the grid and avoids detection in most episodes. The 60% success rate reflects the model's late-stage stalling in some of the more complex maps.

Discussions

Challenges and Solutions

As mentioned in our observation and reward sections, we initially attempted to construct a complete observation state and reward scheme which could complete medium to hard maps. We devised methods for this and modified both to attempt to complete maps, with a particular focus on *maze*, however quickly realized we couldn't solve maps as simple as *just_go*. Realizing this approach wasn't sufficient, we focused on building a scheme which could achieve our first mission success on *just_go*. This is where the baseline series of observation and reward schemes came in, and after successfully completing *just_go*, we moved onto *safe*, then *maze* and finally *chokepoints*. At each stage we encountered new difficulties. The most common was stalling, when the agent would go in small loops or

fully stop moving. This can often be explained by the agent finding local optimums rather than taking a risk and attempting full coverage. Most of the reward scheme's attributes play a role in fighting against this behaviour. When we reached *safe*, the agent had a hard time with corners and smaller nooks, so when we implemented *BASELINE_OBS_V2*, we included a metric showing how many uncovered cells were in each direction which seemed to resolve the issue. The next main issue was enemy navigation, on *maze*, the agent would either be caught or stall immediately before passing the enemy. This was primarily resolved by our runtime features such as a forecasting enemy FOV, implemented in *BASELINE_OBS_V4*. We found a lot of success in modifying the observation space throughout development. However, even with these improvements, the agent would still occasionally exhibit inefficient behavior in late stages of coverage, particularly when only a few cells remained. This suggests that while our methods improved general navigation and survival, fine-grained decision making near completion remains a key challenge.

During development we had two major hiccups. First, we had failed to remember we couldn't modify the env.py file, and began changing the file to add more features to the observation space. After realizing our error, we rerouted these features through the runtime mechanic which has been detailed above. This rework is mildly degraded from the features implemented in the env.py file, as the enriched features are inferred from available information. Secondly, we had a longstanding error in our custom.py file, where the reward function was hardcoded to use our old *COVERAGE_REWARD* scheme even when we told the program to use our newer baseline reward series. This was identified after a large chunk of training and exploration. After discovering this we had to go back and redo testing as we believe most training was done with this error in place. This was a simpler fix than the runtime rework, but set us back in testing our reward functions.

Future Development

The primary goal for future development would be to further improve our best model, such that it could beat the "custom_challenge", "double_maze", "triple_patrol" and "sneaky_enemies" maps. Our current best model can achieve good coverage on these maps, but stalls before it can complete them. To accomplish this, we would need a model which encourages riskier behaviour, to prevent this late-stage stalling, while still maintaining a strong general performance. This late-stage stalling often occurs when only a small number of cells remain, suggesting that the agent struggles to efficiently locate and prioritize unexplored regions.

One possible method for finding this solution would be to develop a separate algorithm for optimizing the reward function. Because there are two, somewhat contradictory, objectives for the reward (risky behaviour vs. general performance) and a huge number of options for testing, this would be well suited to an evolutionary algorithm like NSGA-II (Non-dominated Sorting Genetic Algorithm II). The use of an evolutionary algorithm will also incorporate novelty into the process, providing the opportunity to explore reward functions that might not have otherwise occurred to us. Using an algorithm like NSGA-II would, ideally, present us with more effective reward functions for these difficult maps, without sacrificing our existing performance.

GenAI Disclosure

In accordance with the course syllabus' Use of Generative Artificial Intelligence (GenAI) Tools, we used generative AI during the code development phase of this project. This was used in our train_sb3.py, watch_model.py, custom.py, and plot_experiments.py files to generate the code based on the structure and approaches we described. The bulk of this code was generated using OpenAI's Codex, in particular using the free version of GPT-5.4. Examples of use include:

- Reworking env.py changes to be implemented in custom.py via the runtime mechanic.
- Structuring our train_sb3.py file to make use of gymnasium and the provided env.py file, and saving models and logs for later use.
- Structuring our custom.py file, organizing methods for us to build reward schemes and observation states.
- Implementing watch_model using the provided render function.
- Implementing our plot_experiments.py file to pull trained models and logs to generate informative graphs.

We affirm our use of these tools was in accordance with the guidelines outlined in the course syllabus.

References

K. Deb, A. Pratap, S. Agarwal and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," in IEEE Transactions on Evolutionary Computation, vol. 6, no. 2, pp. 182-197, April 2002.